Конфигурация, Docker и health-check

Разложу по шагам: `.env` → настройки → health → Dockerfile → compose → проверка.

---

1. `.env` — все параметры запуска

Создайте в корне проекта файл `.env` (в репозиторий кладём `.env.example`, сам `.env` — в `.gitignore`).

`.env.example`

dotenv

```
# ─── Application ──────────────────────────────────────────────
APP_NAME=ServiceDesk
APP_VERSION=1.0.0
APP_ENV=development
DEBUG=true
LOG_LEVEL=INFO

# ─── HTTP ─────────────────────────────────────────────────────
APP_HOST=0.0.0.0
APP_PORT=8000
WORKERS=2

# ─── Database ─────────────────────────────────────────────────
# SQLite (по умолчанию) — путь к файлу внутри контейнера
DATABASE_URL=sqlite:////data/servicedesk.db
# Пример для PostgreSQL (если раскомментировать — потребуется драйвер)
# DATABASE_URL=postgresql+psycopg://servicedesk:servicedesk@db:5432/servicedesk
DB_ECHO=false

# ─── Security ─────────────────────────────────────────────────
# Сгенерируйте: python -c "import secrets;print(secrets.token_urlsafe(48))"
SECRET_KEY=change-me-in-production
TOKEN_TTL_HOURS=12
PBKDF2_ITERATIONS=200000

# ─── Seed (dev only) ──────────────────────────────────────────
# Пароли для seed-пользователей из seed.sql, проставляются на старте
SEED_PASSWORDS=true
SEED_CLIENT_PASSWORD=client123
SEED_OPERATOR_PASSWORD=operator123

# ─── Health / readiness ───────────────────────────────────────
HEALTH_CHECK_DB=true
```

`.gitignore` (дополнение)

gitignore

```
.env
*.db
*.db-wal
*.db-shm
```

---

2. Настройки через Pydantic Settings

`app/config.py`

python

```
from functools import lru_cache
from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
        case_sensitive=False,
    )

    # application
    app_name: str = Field(default="ServiceDesk", alias="APP_NAME")
    app_version: str = Field(default="1.0.0", alias="APP_VERSION")
    app_env: str = Field(default="development", alias="APP_ENV")
    debug: bool = Field(default=False, alias="DEBUG")
    log_level: str = Field(default="INFO", alias="LOG_LEVEL")

    # http
    app_host: str = Field(default="0.0.0.0", alias="APP_HOST")
    app_port: int = Field(default=8000, alias="APP_PORT")
    workers: int = Field(default=1, alias="WORKERS")

    # database
    database_url: str = Field(default="sqlite:///./servicedesk.db",
alias="DATABASE_URL")
    db_echo: bool = Field(default=False, alias="DB_ECHO")

    # security
    secret_key: str = Field(default="dev-secret-change-me", alias="SECRET_KEY")
    token_ttl_hours: int = Field(default=12, alias="TOKEN_TTL_HOURS")
    pbkdf2_iterations: int = Field(default=200_000, alias="PBKDF2_ITERATIONS")

    # seed
    seed_passwords: bool = Field(default=False, alias="SEED_PASSWORDS")
    seed_client_password: str = Field(default="client123",
alias="SEED_CLIENT_PASSWORD")
    seed_operator_password: str = Field(default="operator123",
alias="SEED_OPERATOR_PASSWORD")

    # health
    health_check_db: bool = Field(default=True, alias="HEALTH_CHECK_DB")


@lru_cache
def get_settings() -> Settings:
    return Settings()
```

Подключите настройки в `database.py` и `security.py`, чтобы они не читали константы напрямую.

`app/database.py` (обновлённый)

python

```
from sqlalchemy import create_engine, event
from sqlalchemy.orm import sessionmaker, DeclarativeBase

from app.config import get_settings

settings = get_settings()

engine = create_engine(
    settings.database_url,
    connect_args={"check_same_thread": False} if
settings.database_url.startswith("sqlite") else {},
    echo=settings.db_echo,
    future=True,
)

@event.listens_for(engine, "connect")
def _set_sqlite_pragma(dbapi_connection, _):
    if settings.database_url.startswith("sqlite"):
        cur = dbapi_connection.cursor()
        cur.execute("PRAGMA foreign_keys = ON")
        cur.execute("PRAGMA journal_mode = WAL")
        cur.close()

SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False,
future=True)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

`app/security.py` (обновлённый — только заголовок)

python

```
import hashlib, hmac, secrets
from datetime import datetime, timedelta
from app.config import get_settings

settings = get_settings()
SECRET_KEY = settings.secret_key
PBKDF2_ITERATIONS = settings.pbkdf2_iterations
TOKEN_TTL_HOURS = settings.token_ttl_hours

# дальше hash_password / verify_password / create_token / parse_token
# используют эти значения вместо жёстких констант
```

`requirements.txt`

txt

```
fastapi==0.115.5
uvicorn[standard]==0.32.1
```

```
sqlalchemy==2.0.36
pydantic[email]==2.10.3
pydantic-settings==2.6.1
python-dotenv==1.0.1
```

---

3. Health-check endpoint

Требование: «сообщает, что приложение работает». Добавим `GET /health` (liveness) и, опционально, `GET /health/ready` (readiness — с проверкой БД), чтобы Docker/K8s могли различать «процесс жив» и «может принимать трафик».

`app/controllers/health.py`

python

```
from datetime import datetime, timezone
from fastapi import APIRouter, Depends, Response, status
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.config import get_settings
from app.database import get_db

router = APIRouter(tags=["health"])
settings = get_settings()

_started_at = datetime.now(timezone.utc)


@router.get("/health")
def health():
    """
    Liveness: приложение запущено и отвечает на HTTP.
    Никогда не трогает БД — чтобы не падать при её временной недоступности.
    """
    return {
        "status": "ok",
        "service": settings.app_name,
        "version": settings.app_version,
        "env": settings.app_env,
        "uptime_seconds": int((datetime.now(timezone.utc) -
_started_at).total_seconds()),
    }


@router.get("/health/ready")
def readiness(response: Response, db: Session = Depends(get_db)):
    """
    Readiness: приложение готово принимать трафик — БД отвечает.
    Возвращает 200, если всё ок, 503 — если зависимость недоступна.
    """
    db_ok = True
    db_error: str | None = None

    if settings.health_check_db:
        try:
            db.execute(text("SELECT 1"))
        except Exception as exc:  # noqa: BLE001
            db_ok = False
```

```
            db_error = str(exc)

    if not db_ok:
        response.status_code = status.HTTP_503_SERVICE_UNAVAILABLE

    return {
        "status": "ok" if db_ok else "degraded",
        "service": settings.app_name,
        "version": settings.app_version,
        "checks": {
            "database": {"ok": db_ok, "error": db_error},
        },
    }
```

`app/main.py` (обновлённый)

python

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

from app.config import get_settings
from app.database import Base, engine
from app.exceptions import DomainError
from app.controllers import auth, comments, health, tickets, users

settings = get_settings()
Base.metadata.create_all(bind=engine)

app = FastAPI(title=settings.app_name, version=settings.app_version)

@app.exception_handler(DomainError)
async def domain_error_handler(_: Request, exc: DomainError):
    return JSONResponse(status_code=exc.status_code, content={"error":
exc.message})

app.include_router(health.router)
app.include_router(auth.router)
app.include_router(tickets.router)
app.include_router(comments.router)
app.include_router(users.router)
```

Проверка:

bash

```
curl -s http://127.0.0.1:8000/health | python -m json.tool
#
{"status":"ok","service":"ServiceDesk","version":"1.0.0","env":"development","up
time_seconds":5}

curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/health/ready
# 200
```

---

4. Dockerfile

Многоступенчатая сборка, non-root пользователь, healthcheck через `/health`, запуск uvicorn с настройками из `.env`.

Dockerfile

dockerfile

```
# syntax=docker/dockerfile:1.7

# ---------- builder ----------
FROM python:3.12-slim AS builder

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /build

RUN apt-get update \
 && apt-get install -y --no-install-recommends build-essential \
 && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip wheel --wheel-dir /wheels -r requirements.txt


# ---------- runtime ----------
FROM python:3.12-slim AS runtime

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PYTHONPATH=/app \
    PIP_NO_CACHE_DIR=1

# непривилегированный пользователь
RUN groupadd --system app && useradd --system --gid app --home /app app

WORKDIR /app

# зависимости из builder
COPY --from=builder /wheels /wheels
RUN pip install --no-index --find-links=/wheels /wheels/*.whl \
 && rm -rf /wheels

# код приложения и SQL-скрипты
COPY app ./app
COPY schema.sql seed.sql ./
COPY docker-entrypoint.sh /usr/local/bin/docker-entrypoint.sh
RUN chmod +x /usr/local/bin/docker-entrypoint.sh \
 && mkdir -p /data \
 && chown -R app:app /app /data

USER app

EXPOSE 8000

HEALTHCHECK --interval=15s --timeout=3s --start-period=10s --retries=3 \
  CMD python -c "import urllib.request,sys; \
    r=urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2); \
    sys.exit(0 if r.status==200 else 1)"

ENTRYPOINT ["/usr/local/bin/docker-entrypoint.sh"]
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

`docker-entrypoint.sh`

Инициализирует БД (схема + seed + пароли) при первом старте и запускает команду.

bash

```
#!/usr/bin/env bash
set -euo pipefail

DB_PATH="${DATABASE_URL#sqlite:///}"
DB_PATH="${DB_PATH#/}"       # убираем ведущий слэш для относительных путей

# если БД SQLite и файла нет — применяем schema.sql + seed.sql
if [[ "${DATABASE_URL}" == sqlite* ]]; then
  # абсолютный путь вида /data/servicedesk.db → проверим его
  ABS_PATH="/${DB_PATH}"
  if [[ ! -f "${ABS_PATH}" ]]; then
    echo "[entrypoint] initializing SQLite at ${ABS_PATH}"
    mkdir -p "$(dirname "${ABS_PATH}")"
    python - <<'PY'
import os, sqlite3, pathlib
db_path = os.environ["DATABASE_URL"].replace("sqlite:///", "")
db_path = "/" + db_path.lstrip("/")
pathlib.Path(db_path).parent.mkdir(parents=True, exist_ok=True)
conn = sqlite3.connect(db_path)
for f in ("schema.sql", "seed.sql"):
    conn.executescript(pathlib.Path(f).read_text())
conn.commit()
conn.close()
print("schema+seed applied")
PY
  fi

  # при необходимости проставляем пароли seed-пользователям
  if [[ "${SEED_PASSWORDS:-false}" == "true" ]]; then
    echo "[entrypoint] seeding passwords for demo users"
    python - <<'PY'
import os, sqlite3
from app.security import hash_password

db_path = os.environ["DATABASE_URL"].replace("sqlite:///", "")
db_path = "/" + db_path.lstrip("/")
conn = sqlite3.connect(db_path)
pairs = {
    "ivanov@company.ru":    os.environ.get("SEED_CLIENT_PASSWORD", "client123"),
    "sidorov@company.ru":   os.environ.get("SEED_CLIENT_PASSWORD", "client123"),
    "petrova@company.ru":   os.environ.get("SEED_OPERATOR_PASSWORD",
"operator123"),
    "kuznetsova@company.ru":os.environ.get("SEED_OPERATOR_PASSWORD",
"operator123"),
}
for email, pwd in pairs.items():
    conn.execute("UPDATE users SET password_hash=? WHERE email=?",
(hash_password(pwd), email))
conn.commit()
conn.close()
print("passwords seeded")
PY
  fi
fi

exec "$@"
```

---

5. docker-compose.yml

Два варианта — «по умолчанию SQLite» и «с PostgreSQL». Оба файла взаимоисключающие; выберите один.

`docker-compose.yml` (SQLite, простой)

yaml

```
services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    image: servicedesk:local
    container_name: servicedesk-app
    env_file:
      - .env
    environment:
      DATABASE_URL: sqlite:////data/servicedesk.db
      APP_HOST: 0.0.0.0
      APP_PORT: 8000
    ports:
      - "${APP_PORT:-8000}:8000"
    volumes:
      - servicedesk-data:/data
    healthcheck:
      test: ["CMD", "python", "-c",
             "import urllib.request,sys; sys.exit(0 if
urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2).status==200
else 1)"]
      interval: 15s
      timeout: 3s
      retries: 3
      start_period: 10s
    restart: unless-stopped

volumes:
  servicedesk-data:
```

`docker-compose.postgres.yml` (PostgreSQL, overlay)

yaml

```
services:
  db:
    image: postgres:16-alpine
    container_name: servicedesk-db
    environment:
      POSTGRES_DB: servicedesk
      POSTGRES_USER: servicedesk
      POSTGRES_PASSWORD: servicedesk
    volumes:
      - servicedesk-pg:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U servicedesk -d servicedesk"]
      interval: 5s
      timeout: 3s
      retries: 10
    restart: unless-stopped
```

```
  app:
    build:
      context: .
      dockerfile: Dockerfile
    image: servicedesk:local
    container_name: servicedesk-app
    env_file:
      - .env
    environment:
      DATABASE_URL:
postgresql+psycopg://servicedesk:servicedesk@db:5432/servicedesk
      SEED_PASSWORDS: "true"
    ports:
      - "${APP_PORT:-8000}:8000"
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "python", "-c",
             "import urllib.request,sys; sys.exit(0 if
urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2).status==200
else 1)"]
      interval: 15s
      timeout: 3s
      retries: 3
      start_period: 15s
    restart: unless-stopped

volumes:
  servicedesk-pg:
```

> Для PostgreSQL нужны `psycopg[binary]` в `requirements.txt` и миграции (Alembic) — seed-скрипт из entrypoint рассчитан на SQLite. Для приёмки текущего задания достаточно SQLite-варианта; PG-вариант — как задел на прод.

---

6. Запуск и проверка

6.1 Локально без Docker

bash

```
cp .env.example .env
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Проверка:

bash

```
curl -s http://127.0.0.1:8000/health | python -m json.tool
curl -s -o /dev/null -w "ready: %{http_code}\n"
http://127.0.0.1:8000/health/ready
```

6.2 Docker (SQLite)

bash

```
cp .env.example .env
# (опционально) поменяйте SECRET_KEY и пароли
docker compose up --build -d

# статус
docker compose ps
# servicedesk-app  ...  Up (healthy)

# логи
docker compose logs -f app

# проверки
curl -s http://127.0.0.1:8000/health | python -m json.tool
curl -s -o /dev/null -w "ready: %{http_code}\n"
http://127.0.0.1:8000/health/ready
```

Ожидаемый `docker compose ps`:

text

```
NAME               IMAGE                STATUS
servicedesk-app    servicedesk:local    Up 20 seconds (healthy)
```

Docker сам вызывает `HEALTHCHECK` каждые 15 сек. Если `/health` не отвечает — контейнер переходит в `unhealthy`, и `restart: unless-stopped` не спасает (Docker не перезапускает по healthcheck без `autoheal`); для прода добавьте `autoheal` или внешний supervisor.

6.3 Healthcheck с проверкой БД (негативный кейс)

bash

```
# останавливаем БД, если используете postgres-compose
docker compose -f docker-compose.yml -f docker-compose.postgres.yml stop db

curl -s -o /dev/null -w "ready: %{http_code}\n"
http://127.0.0.1:8000/health/ready
# ready: 503
```

6.4 Полный сценарий через Docker

bash

```
BASE=http://127.0.0.1:8000

# health
curl -s $BASE/health | python -m json.tool

# логин оператора (пароль из .env, проставлен entrypoint'ом)
OP_TOKEN=$(curl -s -X POST $BASE/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"petrova@company.ru","password":"operator123"}' \
  | python -c "import sys,json;print(json.load(sys.stdin)['access_token'])")

# поиск
curl -s "$BASE/tickets/search?q=принтер" -H "Authorization: Bearer $OP_TOKEN" |
python -m json.tool
```

6.5 Остановка и очистка

bash

```
docker compose down            # остановить, данные в volume остаются
docker compose down -v         # + удалить volume (БД)
docker compose logs --tail=100 app
```

---

7. Что куда переехало

Было в коде	Стало в `.env`
`DATABASE_URL = "sqlite:///./servicedesk.db"` в `database.py`	`DATABASE_URL`
`SECRET_KEY = "dev-secret-change-me"` в `security.py`	`SECRET_KEY`
`PBKDF2_ITERATIONS = 200_000`	`PBKDF2_ITERATIONS`
`timedelta(hours=12)` в `create_token`	`TOKEN_TTL_HOURS`
`title="ServiceDesk", version="1.0.0"` в `main.py`	`APP_NAME`, `APP_VERSION`
`--host 0.0.0.0 --port 8000` в команде uvicorn	`APP_HOST`, `APP_PORT` (используется в compose/CMD)
жёсткие пароли в скрипте seed	`SEED_CLIENT_PASSWORD`, `SEED_OPERATOR_PASSWORD`
включение `echo=True` для отладки SQL	`DB_ECHO`

8. Чек-лист

- □ `.env.example` в репозитории, `.env` — в `.gitignore`
- □ `app/config.py` читает все параметры, `lru_cache` на `get_settings()`
- □ `database.py`, `security.py`, `main.py` используют `get_settings()`
- □ `GET /health` — 200 всегда, когда процесс жив
- □ `GET /health/ready` — 200 при доступной БД, 503 при недоступной
- □ `Dockerfile` — multi-stage, non-root, `HEALTHCHECK` на `/health`
- □

`docker-entrypoint.sh` — идемпотентно инициализирует SQLite (schema + seed + пароли)

- □

  `docker-compose.yml` — `env_file`, volume на `/data`, healthcheck, `restart: unless-stopped`

- □

  `docker compose up --build -d` → контейнер `healthy`, `curl /health` → `status: ok`

- □

  Секреты (`SECRET_KEY`, пароли БД) не зашиты в образ — только через env